Agent Design -- SIMS

Student: Marcuel L. Gubaynon

Course: Laravel Web Development

Project: Student Information Management System

Prepared: 12 November 2026

1. Proposed specialized workflow

1. The student developer sets the scope, reviews changes, and explains the final decisions.

2. The Planner turns the handbook requirement into a small task and acceptance checks.

3. The Database role handles schema changes when needed; the Laravel role implements routes, validation, controllers, and views.

4. The Testing role checks behavior and authorization. The Reviewer examines the diff and reports risks.

5. The Documentation role updates reports and instructions from verified results.

6. The student accepts the change and demonstrates it.

The Database, Laravel, and Documentation roles can work in parallel only after the Planner defines clear boundaries. Testing and review follow implementation.

This workflow defines the project's role design. The role prompts can be assigned to separate agents when delegation is available, or used one at a time as checklists.

2. Role responsibilities and handoffs

Role | Reads | Produces | Does not do

Planner | AGENTS.md, handbook/rubric, current feature summary | Small plan and observable acceptance checks | Write application code

Database | Migrations, models, feature plan | Migration/model changes and schema handoff | Change controllers/views; rewrite migrations that may already have run

Laravel | Schema handoff, routes, requests, relevant views/tests | One feature implementation using MVC and existing authorization | Change schema without a handoff; put business logic in Blade

Testing | Routes, controllers, Form Requests, current tests | Feature tests and validation/security findings | Change app code or weaken tests

Reviewer | Diff, AGENTS.md, acceptance checks | Severity-ordered findings with file/line and correction | Apply unreviewed changes silently

Documentation | Verified feature and actual prompt/check records | README/API/prompt/verification updates | Invent screenshots, model names, features, or test results

Standard handoff format

- Feature and acceptance checks

- Files/routes/tables changed

- Validation and role boundaries

- Exact checks run and their results

- Remaining manual checks or risks

- Next role and the concise context it needs

Role instructions are stored in agents/planner.md, database.md, laravel.md, testing.md, reviewer.md, and documentation.md.

3. Context management

Start each task with AGENTS.md and one rubric requirement. Give each role only the relevant feature files. For example, a student API task needs the API routes/controller/resource, student Form Request/model, auth middleware, and API tests. A schema task needs the related migrations and models. Pass a short schema/route summary between roles.

Leave out .env, credentials, vendor/, generated caches, the full database, and unrelated feature folders. For a production bug, provide the exact error text, route/action, and environment. See docs/CONTEXT_STRATEGY.md for the repeatable rules and reflection prompts.

4. Task decomposition and token management

1. Split a broad request into a small feature with observable acceptance checks.

2. Reuse stable project rules from AGENTS.md rather than repeating them.

3. Separate schema design, application code, review, testing, and documentation when the feature is broad.

4. Ask for a short plan before coding if a schema or behavior decision is unclear.

5. Reuse concise handoffs rather than resending the entire project.

6. Review and verify each feature before stacking another change on top.

The tradeoff is to keep prompts small while retaining enough schema, route, and authorization context to avoid breaking adjacent features.

5. What was actually used

The history in docs/PROMPT_LOG.md usually records Primary assistant; no subagent. It documents AI-assisted iterative work, but it does not establish that these specialized roles ran as separate agents. This report therefore presents the role architecture as a designed workflow, not as a false execution claim. If the student later runs actual specialized agents, record their task, inputs, outputs, and handoffs in the prompt log.

6. Student reflection questions

Before final submission, add a specific example for each question:

1. Which file subset or summary did you give an AI for one feature?

2. Which refinement changed the result, and why?

3. What context did you deliberately leave out?

4. How did you verify generated output, and what remained unverified?

5. Which specialization would have helped most, and which would have been unnecessary for this project?